

Machine Learning for Exoplanet Detection: Identifying Exoplanets in Light Curves

Tristan Moffett, Josh Manchester, Brianne Leatherman

University of Colorado at Colorado Springs

Abstract

The discovery of exoplanets has become more dependent on machine learning as missions like Kepler and TESS generate thousands of transit signals that are too costly and time consuming to vet by hand. In this research, we address this problem by comparing three different deep learning models for exoplanet classification: Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Transformer architecture. Each team member designed, trained, and evaluated one of these models using observational data from Kepler and the Transiting Exoplanet Survey Satellite (TESS) along with simulated datasets. Our focus is on performance metrics such as accuracy, precision, recall, F1 score, and Area Under Curve (AUC), as well as understanding the strengths and weaknesses of each model in classifying planets versus false positives.

This paper presents preliminary results from the BiLSTM + clustering approach, which achieved an AUC of 0.6947 on validation data (655 windows with 150 positive examples) and successfully identified TIC 307210830, a confirmed multi-planet system from TESS, demonstrating real-world applicability. The RNN implementation combines K-means clustering on Box Least Squares features with a 3-layer bidirectional LSTM architecture, enabling the network to learn cluster-specific patterns and distinguish different types of stellar variability from genuine planetary transits. The model uses class weighting to handle the 23 percent positive class imbalance and achieves meaningful predictive power despite the small training dataset. From analyzing results across the three architectures, we aim to identify which methods are most effective for generalizability across multiple datasets and advance automated exoplanet vetting for future missions.

Introduction & Background

In the last twenty years, there have been significant increases in the number of space missions whose goal is to observe distant stars and the changes that may occur over variable lengths of time. NASA has been one of the primary drivers behind these advances, which are called transit survey missions. Of most importance to our research are two

missions in particular: Kepler/K2 and TESS. Kepler was launched in 2009 with the original mission of monitoring a 116 square degree segment of sky for three and a half years, and when the satellite had a mechanical issue, the mission was changed to K2, which monitored stars and systems along the elliptic plane until it was retired in 2018 (?). The Transiting Exoplanet Survey Satellite (TESS) was launched in 2018, nearly ten years after Kepler, with the goal of monitoring a sector of approximately 2300 square degrees for about 27 days before moving to the next section (?). At the time of this writing, these two transit surveys have accounted for more than 3000 of the 6000 total confirmed exoplanets (?).

The most prevalent method of determining the presence of an extra-orbital planet has been through observing changes in the intensity of the brightness of a star when an object transits between the star and the observer. These transits can be found by representing the observed flux from the star in a time-series graph, also known as light curves, and then analyzing the data (?). This process often includes phase-folding the data by overlapping periods to determine patterns (?). A dip in the light curve indicates a threshold crossing event (TCE) but that alone does not mean a planet caused the obstruction of the light. The TCE can also be a false positive that was triggered by an eclipsing binary or instrumental artifact (?).

The similar nature of the light curve of a transiting exoplanet and that of an eclipsing binary can be difficult to differentiate. Additionally, light curves can be challenging to work with due to the continuous nature of background noise from instrumental, photon, and other interference in the transit of light, and the curves can require significant work to pre-process and then analyze the data (?). As deep learning continues to develop, the use of neural networks has become a common tool in this process, significantly reducing the number of light curves that need to be analyzed by hand.

The Convolutional Neural Network (CNN) was introduced as a model designed to detect repeating patterns in data. For exoplanet research, this means recognizing the U-shaped dips of planetary transits and separating them from the V-shaped dips of eclipsing binaries within time-series light curve data. A challenge for CNNs is that light curves often contain a large amount of noise, which can make train-

ing more difficult. However, when handled properly, CNNs can still learn to use this noise to improve classification. (?) demonstrated the value of CNNs on Kepler data by combining global and local views of a light curve to capture both the orbital view and the details of a transit. (?) later applied a similar approach to TESS data and introduced additional features such as centroid and stellar parameters to handle noisier signals. These results show why CNNs are a strong fit for exoplanet classification, though other deep learning approaches like Recurrent Neural Networks (RNNs) and Transformers bring their own strengths for working with time-series data such as light curves.

Recurrent Neural Networks (RNNs) are a natural fit for time-series data because they learn patterns across many time steps and keep context memory (history) from earlier points in the sequence. In light-curve analysis, this lets the model use both the local transit shape of the light curve and the surrounding temporal pattern (periodic repeats) at the same time. Gated variants like LSTM and GRU help with long-range dependencies and noisy sequences of data. Stacked LSTMs have shown to be a strong resource for space-mission photometry: for example, an LSTM detector trained on Kepler flares generalized well to TESS with high precision/recall when preprocessing and class balancing were handled carefully (?). Beyond direct classification, sequence-aware representation learning with an Echo State Network (ESN) + autoencoder reconstructs light curves in sequence space (not just a readout vector), producing more faithful time-series embeddings for future tasks (?). Finally, RMTTP shows how an RNN can encode event history to predict both the timing and type of the next event; adding a lightweight timing cue (intervals/slight inaccurate period) alongside the classifier can reduce aperiodic false positives in transit searches (?). In this project, we will benchmark LSTM/GRU on our curated subset, report precision, recall, F1, and ROC-AUC, and optionally include a small timing helper to capture periodicity.

The Transformer architecture was introduced as a novel, simple network relying on encoder/decoder layers of self-attention mechanisms (?). Because attention can be calculated in parallel, this network saw an improvement in quality of results with a reduced amount of training time compared to prior RNN and CNN approaches (?). Although Transformers were originally designed for Natural Language Processing (NLP), their use in other fields has rapidly spread, due to both the efficiency and the ability to correlate sequential data like time series such as a light curve, where the transit of a planet can be considered a function of time (?). Given the rapid advancements in deep learning methods over the last decade, a direct performance comparison across RNNs, CNNs, and Transformers on light curve analysis should prove interesting. This paper will detail the methods for creating such a comparison, as well as results, by modeling multiple research processes from other papers.

Related Work

Convolutional Neural Networks (CNNs)

Shallue and Vanderburg (?) addressed the problem of classifying Threshold Crossing Events (TCEs) from the Kepler mission, where thousands of signals had to be examined and most turned out not to be planets. To resolve this bottleneck, they designed one of the first convolutional neural networks (CNNs) for exoplanet detection, comparing a linear model, a fully connected network, and a 1-D CNN, with the CNN proving most successful. Their final architecture called AstroNet processed two versions of each light curve: a global view covering the full orbital phase and a local view zoomed in around the transit. Each view passed through layers of one-dimensional convolutions and max pooling before being merged into fully connected layers of 512 nodes, with dropout and data augmentation used to reduce overfitting. On Kepler’s real observational test set the CNN achieved results with 96% accuracy, 98.8% AUC, 96% precision, and 95% recall. However, when evaluated on simulated datasets the model struggled, recovering 81% of true injected planets but incorrectly labeling around 90% of background planets and eclipsing binaries as planets, misclassifying about 65% of eclipsing binaries, and letting through 3.18% of false positives from inverted light curves. These outcomes demonstrated that while the model was powerful on Kepler data, it lacked generalizability, leading the authors to suggest training with simulated data and including centroid information to better handle false positives. This highlights the need for models that can work across both real and simulated datasets, a direction we plan to take in our own work.

Dattilo et al. (?) focused on the challenge of vetting the large number of planet candidates identified in K2 light curves, which are especially noisy due to pointing instabilities from the Kepler telescope. This created difficulty in classifying true planet signals from false positives such as eclipsing binaries or systematics. To address this, they extended the AstroNet CNN first built for Kepler data by Shallue and Vanderburg (2018), adapting it to the shorter and noisier K2 campaigns. Their approach used both real K2 TCEs labeled as candidates, binaries, or junk, and tested on simulated “Kepler-lite” data to tune their model under similar conditions. Each TCE was represented by two views of the light curve, a global view showing the full orbital phase and a local view zoomed in on the transit. These were normalized and then passed through separate convolutional and max pooling layers before being merged into fully connected layers of 512 nodes, with additional scalar features such as planet-to-star radius ratio and impact parameter included to strengthen classification. The final output gave the probability of a candidate being a true planet. Their CNN achieved high accuracy averaging 98% on test sets and successfully uncovered two new super-Earths within the K2 dataset. This work highlights the need for both global and local inputs including additional scalar features as it improves against noisy data while also improving the classification of planets and false positives.

Osborne et al. (?) addressed the challenge of rapidly classifying the thousands of planet candidates generated

monthly by TESS. To further the issue each month around 500 hours were required to select the best candidates from each month's findings. As any sane person would be able to tell, this is a bottleneck as it requires too many resources and people that could be used elsewhere. To resolve this issue however, Osborne and his colleagues trained a CNN model based on Shallue and Vanderburg (2018) and Ansdell et al. (2018) work, specifically the AstroNet and ExoNet CNN models. They trained and validated their CNN strictly on high-fidelity simulated TESS light curves (the TSOP-301 dataset), which provided known ground truths for both planets and false positives. Each candidate was represented by both a local and global view of its light curve, with centroid information also included to help spot background eclipsing binaries. Alongside this, during training, they included balanced batch sampling to counter class imbalance (only 14% of candidates were true planets) and applied augmentation techniques to improve generalization and reduce overfitting. When tested, their model achieved strong results on their simulated set, with a 97% average precision in binary classification and a 92% accuracy in identifying planets in their multi-class classification. However, when using the real TESS set, the model's performance dropped, only being able to recover 61% TOIs (TESS Objects of Interest) as planets. Though the model was still able to flag around 200 new TCEs as possible planets. Results highlight a reduction in human workload and planet discovery but also show a gap between simulated and real data performance in CNNs. This brings a need to build a model that is more generalizable capable of handling both simulated and observational dataset together, which we plan to explore in our project.

CNN Classifier for Next Generation Transit Survey

Chaushev et al. (?) focused on improving the speed and consistency of exoplanet detection in large transit surveys, particularly the Next Generation Transit Survey (NGTS), where manual vetting of light curves is reliable but extremely time-consuming. To automate this process, the authors introduced a convolutional neural network (CNN) called PlaNET, which aimed to reduce dependence on large labeled datasets by using a mix of real NGTS observations, unlabeled data, and simulated planetary transits. This combination of data helped the model learn from both real and synthetic signals while improving generalization to noisy and incomplete data. Each light curve was detrended, normalized, and phase-folded so that transits aligned, creating a consistent input for classification. Their architecture was inspired by AstroNet, featuring both global and local views of the light curve to capture the full orbital shape and transit depths. Additional stellar information, such as radius and orbital period, was included to help the model separate planets from false positives like eclipsing binaries or background blends. The model was trained using the Adam optimizer with early stopping and 10-fold cross-validation to avoid overfitting. In testing, PlaNET achieved around 96% accuracy and an AUC of 99.3%, successfully recovering 13 out of 14 confirmed NGTS planets while identifying several new candidates missed by human vetting. Their results show that CNNs can greatly reduce manual effort and improve the reli-

ability of large-scale exoplanet searches. The main takeaway here is the use of extra features such as the radius and orbital period. These features further help in the process of classification, especially in the case where exoplanet lightcurves look similar to EBs and BEBs.

Stella CNN Flare Classifier Feinstein et al. (?) built a CNN called stella to detect stellar flares in TESS short-cadence data. The problem they faced was that manually identifying flares across thousands of young stars is too time-consuming, especially since high stellar variability makes simple outlier detection methods fail. Their solution was to train a CNN to recognize flare shapes directly from the light curves and use this to study how flare rates change with stellar age and temperature. They worked with 3,200 young stars from TESS Sectors 1–20, detrending and normalizing each light curve before splitting them into 200-cadence segments centered on either known flare events or non-flare regions. The architecture used standard components with one-dimensional convolutional layers to extract features, pooling layers to reduce dimensions, fully connected layers with dropout to prevent overfitting, and a final sigmoid layer for binary classification. They trained it with the Adam optimizer and binary cross-entropy loss, running 10 models with different initializations to create an ensemble and reduce overfitting. Their model stella performed really well, hitting an AUC of 0.993 and F1 scores above 0.98 on validation data. When applied to the full sample, it found over 17,000 flares and showed that flare activity drops off significantly for stars older than 50 Myr and hotter than 4000 K. What's interesting here is the data preparation approach. Instead of phase-folding like most exoplanet detection methods, they used fixed-length segments centered on events of interest. This could also work for planet detection, where we would just center segments on threshold crossing events instead of flares.

GPU-accelerated Phase Folding Wang et al. (?) developed GPFC, a pipeline that combines GPU-accelerated phase folding with a CNN to detect ultra-short-period planets with orbital periods less than one day. Most exoplanet detection pipelines rely on Box Least Squares (BLS) to identify transits, but BLS becomes extremely slow when scanning through thousands of trial periods at high resolution. Wang et al. took a different route by offloading the phase folding computation to the GPU and using a CNN to score each folded result. The workflow starts with detrending and outlier removal, then creates a dense grid of trial periods to test. Next, the light curve gets folded and binned at each period, followed by noise normalization to standardize the input. Finally, a CNN evaluates each folded light curve and outputs a score indicating the likelihood of a real transit signal. The impressive part is the speed at which they can fold a single light curve at 100,000 different trial periods in just five seconds on an RTX 3090 Ti. This works by batching 16-period folds simultaneously and using efficient binning algorithms that take advantage of GPU parallelization. The CNN itself has 19 layers and includes circular convolution, global max pooling, and dropout layers to prevent overfitting. They trained it using TensorFlow with the Adam

optimizer. When they tested GPFC against traditional BLS methods, it performed better at detecting low signal-to-noise ratio transits and ran much faster. In a blind search over the 0.2-1.0 day period range, GPFC successfully recovered all confirmed ultra-short-period planets in the Kepler catalog. What makes this approach valuable is that GPU acceleration turns exhaustive period searches from impractical to practical. Testing 100,000 trial periods per light curve would take hours with the usual BLS method, but the GPU implementation cuts this down to seconds. Our current pipeline doesn't use GPU-accelerated folding yet, but this technique could become important if we need to process larger datasets or want to search finer period grids in future work.

Applying These Ideas to Our Work Looking at what worked across these papers, we're going to borrow several techniques that consistently improve detection performance. The dual-view architecture from Shallue and Vanderburg (?) is one of the most useful ideas. It splits each light curve into a global view (2001 bins) covering the full orbital phase and a local view (201 bins) zoomed in on the transit itself. This way, the CNN gets both the orbital context and the finer transit details. We'll follow their preprocessing approach too: detrend with median subtraction, use Box Least Squares to find the period, phase fold around the transit center, and normalize so the baseline sits at zero with the deepest transit point at -1.

The auxiliary scalar features from Dattilo et al. (?) and Chaushev et al. (?) seem worth including as well. Adding things like planet-to-star radius ratio, impact parameter, transit depth, and orbital period helps the model distinguish real planets from eclipsing binaries, especially when the light curves alone are ambiguous or noisy. Osborne et al. (?) showed that centroid information can catch background eclipsing binaries that might otherwise slip through, and their balanced batch sampling approach handles the severe class imbalance problem we're likely to face. We should also test some of the data augmentation techniques that popped up across these studies to make the model more robust.

One issue that kept showing up in all these papers is the generalization problem. Models trained on real data often struggle with simulated datasets and vice versa. To address this, we're planning to train and test on both real observational data (Kepler, K2, TESS) and simulated light curves. That way, we can actually measure how well the model transfers across different missions and figure out what techniques improve cross-dataset performance.

Recurrent Neural Networks (RNNs)

This section reviews RNN applications to astronomical time series that inform the BiLSTM + clustering approach. We first summarize six foundational papers from our midterm report, then present three new studies that establish performance benchmarks and validate key architectural decisions.

Foundational Work (Midterm References) **Vida et al. (2021)** trained stacked LSTMs to detect stellar flares in Kepler and TESS photometry, using dropout regularization and minority class weighting (?). *Why I used it:* Their finding

that models generalize well across Kepler and TESS supports consistent preprocessing, and their comparison showing stacked LSTMs outperform GRUs for astrophysical noise informed my choice of multi-layer BiLSTM architecture.

Kügler et al. (2016) paired Echo State Networks with autoencoders for unsupervised representation learning on Kepler light curves, evaluating reconstructions in sequence space rather than embedding space (?). *Why I used it:* Their emphasis on sequence-level evaluation motivates evaluating my model on full windows and visualizing predictions across time to verify alignment with actual transit events.

Du et al. (2016) introduced Recurrent Marked Temporal Point Processes (RMTTP), demonstrating how RNNs can encode event timing history for predicting when and what type of event occurs next (?). *Why I used it:* This concept motivates incorporating periodicity information—genuine transits repeat on predictable schedules while false positives are aperiodic—through BLS period estimates as clustering features.

Speiser et al. (2020) demonstrated that K-means clustering ($k=5-20$) followed by cluster-aware classification outperforms global classifiers by 8-15% in F1 score on heterogeneous microscopy datasets (?). *Why I used it:* This directly inspired my clustering approach—rather than training one BiLSTM to handle all stellar variability, I cluster windows by BLS features and let the network specialize via learned embeddings.

Vu et al. (2024) validated 3-layer stacked LSTM architectures for time series forecasting, showing 23% improvement over 2-layer variants and 62% F1 improvement from class weighting on imbalanced data (?). *Why I used it:* Their findings directly inform my hyperparameter choices: 3-4 LSTM layers, dropout 0.3-0.4, and inverse class ratio weighting ($\text{pos_weight} = 3.367$).

Ding et al. (2024) applied bidirectional LSTMs to photometric redshift estimation on 4.2 million SDSS galaxies, achieving 34% outlier reduction versus MLP and demonstrating cross-survey transfer learning (?). *Why I used it:* Their success with BiLSTM for astronomical photometry and validation of transfer across surveys (SDSS to DES) supports my architecture choice and suggests Kepler-trained models can adapt to TESS.

New Reference: Variable Star Classification with CNN-LSTM **Becker et al. (2021)** developed a hybrid CNN-LSTM architecture for classifying variable star light curves from OGLE and CRTS surveys, achieving 92-95% accuracy across eclipsing binaries, RR Lyrae, Cepheids, and long-period variables (?).

Architecture. The hybrid combines 1D convolutional layers (kernel sizes 3-7) with LSTM layers. CNNs extract local morphological features (transit shape, ingress/egress profiles) while LSTMs capture long-range temporal dependencies (periodicity, phase relationships). Max pooling between components reduces sequence length for computational efficiency.

Class Imbalance Handling. Their dataset had 10-20× class imbalance between common and rare variable

types. They addressed this through stratified sampling and weighted loss—the same strategy I apply with `pos_weight` for the 23% positive class ratio in transit detection.

Key Finding. The hybrid achieved strong performance using raw photometric time series without hand-engineered features (period, amplitude, phase-folded representations). This end-to-end learning approach validates my decision to feed normalized flux windows directly into the BiLSTM.

Relevance. Two lessons for my implementation: (1) raw time-series input works well, eliminating information loss from feature engineering; (2) weighted loss handles severe class imbalance at 92-95% accuracy, confirming my `pos_weight` approach.

New Reference: Machine Learning for Ground-Based Transit Surveys Schanche et al. (2019) tackled automated vetting of transit candidates from the WASP ground-based survey, achieving approximately 90% correct planet identification using ensemble classifiers (?).

Methodology. They trained both Random Forest classifiers and CNNs on light curves containing planetary transits, eclipsing binaries, variable stars, and non-periodic signals. Individual classifiers produced significant false positives, but ensemble voting reduced errors. This finding resonates with my clustering strategy: rather than forcing one model to handle all variability, clustering enables specialization.

Distinguishing Planets from Binaries. A major focus was separating genuine transits from eclipsing binary false positives, which produce deeper, longer dips and often show secondary eclipses. The authors emphasized transit depth and duration as key discriminating features, directly motivating my inclusion of these BLS features in clustering.

Relevance. Ground-based surveys face much noisier conditions than TESS due to atmospheric interference. If machine learning achieves 90% accuracy on noisy ground-based data, my BiLSTM with cluster embeddings should achieve comparable or better results on cleaner space-based photometry. The emphasis on depth and duration validates my BLS-feature clustering.

New Reference: Feature-Based Detection with Gradient Boosting Malik et al. (2022) took a fundamentally different approach: extracting 789 statistical features using TS-FRESH and feeding them into LightGBM gradient boosting, achieving AUC 0.948 and recall 0.96 on Kepler data (?).

Feature Engineering. TSFRESH automatically extracts time series features including mean, variance, autocorrelation at various lags, spectral properties, and entropy measures. These 789 features describe shape, periodicity, and statistical properties without requiring neural networks.

Key Results. Their feature-based approach correctly ranked planet signals above non-planets 95% of the time and detected 96% of real planets. This remarkably strong performance for a non-deep-learning method challenges neural network approaches to justify their added complexity.

Benchmark Implications. Malik et al.'s 0.948 AUC establishes a concrete target. If my BiLSTM cannot match or exceed this, the complexity may not be justified. However, gradient boosting treats features independently, los-

ing sequential information. My BiLSTM learns temporal patterns—the U-shaped transit profile, phase relationships—that feature-based methods cannot capture. The architecture combines explicit domain features (BLS period, depth, duration for clustering) with learned temporal representations.

Synthesis: Informing the BiLSTM + Clustering Design These nine works collectively inform the architecture and training strategy:

- **Architecture:** 3-4 layer BiLSTM with 256 hidden units (Vida, Vu, Ding, Becker)
- **Clustering:** K-means ($k=5$) on BLS features before training (Speiser, Schanche)
- **Class balancing:** `pos_weight = 3.367` in BCEWithLogitsLoss (Vu, Becker)
- **Input:** Raw normalized flux windows (Becker)
- **Regularization:** Dropout 0.3-0.4 + early stopping (Vida, Vu)
- **Evaluation:** Sequence-level diagnostics (Kügler), timing features (Du)
- **Target:** AUC > 0.948 (Malik)

This design addresses class imbalance through weighted loss, heterogeneous stellar variability through cluster embeddings, and temporal dependencies through bidirectional processing.

Transformer Architecture

Attention mechanisms have been found to perform very well in sequence modeling, allowing models to capture dependencies between tokens regardless of the distance between positions. Self-attention (or intra-attention) creates a relationship between different positions of the input by using an encoder-decoder structure made of stacked layers.

The research paper "Attention Is All You Need" (?), is the first of its kind to fully implement the Transformer architecture, and demonstrates this by training the model on English-to-German and English-to-French translation. Many of the researchers on this project were concurrently working at Google Brain and Google Research. This paper is currently one of the most influential of the 21st century and is listed as the 7th most-cited piece of research in the last 25 years by Nature. Transformer architecture has been the major basis for most Natural Language Processing (NLP) projects as well as generative Large Language Models (LLMs).

?'s design of Multi-Head Attention accomplishes this by projecting queries, keys, and values h times in parallel. This allows the model to attend every token to all the other tokens in the sequence, and weights the positions with the average attention to other tokens, in each of the six layers of the encoder and decoder. Additionally, each head focuses on a different pattern/relationship in the data. The parallelization within the heads helped reduce the sequential limitations of RNNs when operating on larger data sets.

? made two models: a small one and a big one. When trying to test a translation from English to German, the big Transformer model achieved a new high BLEU score of 28.4, surpassing previous models by over 2.0 BLEU, and

was trained on 8 NVIDIA P100 GPUs for 3.5 days. Even the small model, which was only trained for 12 hours, still beat the other models by a smaller margin. The large model also beat all other models when translating English to French, though only by 0.5 BLEU, and the smaller model preformed similarly to the others.

This experiment, though not focused on the interpretation of light curves, gives us an encouraging basis to pursue how effective an Attention-based model could be in analyzing transit data.

This idea was pursued by ? in their ? paper, "Don't Pay Attention to the Noise: Learning Self-supervised Representations of Light Curves with a Denoising Time Series Transformer." In this paper, they designed a Transformer model named the Denoising Time Series Transformer (DTST) to try to remove noise, fill in missing data, and fine-tune the light curves to make it easier to analyze. The architecture uses real stellar light curves from TESS's first sector, showing greater adaptability than other traditional denoising techniques. ? used the same positional embedding as ? (?) to keep track of the chronological sequence of points on the light curve.

After standardizing the light curve data from TESS and training the model, ? found that results would not always denoise noisy data and might overfit quiet data, due to a lack of data normalization. The DTST research explicitly addressed the challenges of light curves, which are noted as "particularly challenging data objects." Although the results were not as consistent as desired, the experiment showed a lot of potential for the continued exploration of building a more robust Transformer model. They additionally found they could track how important the model thought different parts of the data were, by using an attention map.

The work "Distinguishing a planetary transit from false positives: a Transformer-based classification for planetary transit signals," (?), directly continues the search for efficient, interpretable models to classify TCEs and determine the possibility of whether the TCEs were actually exoplanets or a false positive. Other models typically used CNNs with many layers, which could quickly become computationally complex. ?'s classification system was specifically designed to use phase-folded light curves from TESS, as well as flux data and centroid pixel position information, and included extra stellar and transit parameters (like period, duration, and host star temperature). The inputs were embedded into a vector representation using a convolutional layer before going to the multihead attention layer. Much of this preprocessing was handled with the Python packages 'lightcurve' and 'astropy api'. ? ended up scaling the light curves "by removing the mean and scaling to unit variance for both centroids." When training the model, they only used one decoder which performed like an activation function, and three encoders.

Once trained, the Transformer achieved competitive results as other existing CNNs applied to TESS data with a similar goal, with average recall, precision, and F1 scores of 0.88. A major benefit of Transformers is the enhanced interpretation, and that the self-attention mechanism allowed the authors to observe that parameters like transit depth and host star temperature were highly influential in the classification

decision.

Due to the continued results in pursuing Transformers, Salinas came back with another group of researchers to expand upon their previous research. This time the project was "Exoplanet transit candidate identification in TESS full-frame images via a transformer-based algorithm," (?). The research is focused around removing the need for highly pre-processed and phase-folded light curve data before analyzing for potential exoplanet candidates. ? wanted to identify exoplanets that only had a single transit within the data segments (monotransits), which would remove the periodic requirement from the light curve data. Because TESS data is limited to approximately 27 days, requiring a repeating transit will only find planets with orbits that take less than 25 days. This would additionally help identify planets with Transit Timing Variations (TTVs) which could indicate a star system with multiple planets.

This approach continues the classification properties from the previous study (?) and takes inputs of raw flux, centroid data, and background time-series from full-frame TESS images. Test data included labeled light curves that indicated confirmed exoplanets, confirmed eclipsing binaries or other false positives, and non-transit signals. The process begins by finding local features (short-range patterns) with a convolutional embedding layer (similar to (?)), and these tokens are then fed into the multi-headed attention encoder to learn additional patterns.

Because the model can evaluate long-range dependencies, it can identify the characteristic shape of a planetary transit curve even if only a single transit is shows up in the entire 27 days of observation. This is important for detecting monotransits and planets with longer periods.

When the model was tested on brand new TESS data, it found approximately 0.1 percent of the light curves indicated an exoplanet transit. With eight layers of encoding heads, they found an average Area Under Curve (AUC) score of 0.88 and an average F1 score of 0.83.

By the end of the experiment, ? ended up testing sectors 1-26 of TESS data, which included 4.1 million light curves. Again, the model found approximately 0.1 percent to be potential candidates for planet transits. After confirming data through other means of verification, about 25%, or 214, of the candidates were strong exoplanet candidates, including 88 single-transit candidates, which demonstrates the model's capability to detect transit events regardless of periodicity.

When looking at the paper "Transformer Architectures for Time Series Classification" (?), we can see how the utilization of Transformer architectures in time series analysis stems from their fundamental ability to capture long-range dependencies and interactions in sequential data. This is particularly attractive for classification tasks, as Transformers have a prominent capability in capturing long-term dependency necessary for distinguishing sequence categories.

Recent research shows that optimizing the attention mechanism to capture temporal structure is key to superior performance. For instance, the MVGFormer model, which uses a visibility graph-guided attention mechanism (VG-Attention), excels in sequence-level classification (?). In

evaluations against existing methods on 17 multivariate time series datasets from UEA, MVGFormer achieved superior performance and demonstrated a strong representation capability in the vector space, effectively distinguishing between different categories. This success is attributed to its ability to focus attention on specific key time points or time intervals, such as periodic local modules or long-term trending structures, avoiding the dispersed attention characteristic of standard Transformer methods.

While standard Transformer models are adept at modeling global correlations, they often lack inherent assumptions about data locality. Conversely, traditional Convolutional Neural Networks (CNNs) are limited by their localized receptive fields, which makes them inefficient at capturing interactions between long-range information without stacking numerous layers. This limitation has often resulted in astronomic analyses focusing on the frequency domain (power spectra), which discards crucial non-Gaussian phase information present in the time domain.

To overcome these constraints and fully leverage the richness of long time series like stellar light curves, hybrid models that integrate both approaches are essential. The Astroconformer model provides a direct architectural precedent for my project. Astroconformer is a Transformer-based deep learning framework designed to capture both local short-range and global long-range information in stellar light curves, which contain information about oscillations and granulation (?).

The architecture successfully integrates the following components:

1. **Convolutional Modules:** Astroconformer incorporates convolutional modules (including a convolutional layer, batch normalization, and Swish activation) to handle local feature extraction. This convolutional component is crucial because while the multi-head self-attention (MHSA) is theoretically sufficient, incorporating these convolutional layers enhances performance when labeled data is relatively limited.
2. **Self-Attention Layers (MHSA):** The MHSA component is used to capture long-range dependencies.

Ablation studies comparing Astroconformer variants confirm the strategic value of this hybrid approach. The data shows a clear performance decline as the receptive field size diminishes across various models (including pure CNNs like ResNet-18). Specifically, ? found that replacing the MHSA module with a standard linear projection layer led to the most significant performance drop in forecasting tasks for MVGFormer, and in $\log g$ inference for Astroconformer, underscoring the key role of the global attention mechanism in modeling long-range dependencies.

In the context of light curves, this architecture enables the model to leverage information beyond just the oscillations signal, successfully determining parameters like $\log g$ based on granulation patterns, which is critical for dwarf and sub-giant stars where oscillation signals may be brief or unresolved. The model achieved high precision, with an RMSE for $\log g$ of 0.017 dex in dense sampling regimes.

Dataset	Purpose	Notes/SOTA
Simulated Data (~10 known transits)	Train/val	Small, hand-checkable; class balance documented. Data from NASA Exoplanet Archive.
Pseudo/synthetic injections	Stress tests	Inject depth/duration/period jitter; report injection policy. Data from NASA
(Final Test) Kepler/TESS subsets	Generalization and best method determination	Add a SOTA line when datasets are finalized (TBD). Data from MAST and NASA Exoplanet Archive

Table 1: Planned datasets for ML learning and testing.

The initial use of a CNN (or convolutional layer) for input preparation is also recommended. This embedding step aims to extract localized, short-range features before the Transformer analyzes the global sequence. Astroconformer similarly segments the input light curve into patches (e.g., 20 timestamps, representing 10 hours of data) and transforms them into feature vectors using an MLP (which acts as an embedding layer), effectively creating a coarse temporal axis before the encoder extracts local and global features. This is similar to (?) where planetary transits in TESS data used a similar approach in which inputs were embedded into a vector representation using a convolutional layer before entering the multihead attention layer.

Datasets

For this research, we will be using both simulated and labeled observational data. The simulated data is available from the NASA Exoplanet Archive and includes injected on- and off-target transits, scrambled data, and false positives within K2 light curves. The labeled observational data will come from TESS datasets cross-referenced with matching labels, both from the Mikulski Archives for Space Telescopes (MAST). The purpose for choosing the above datasets is it provides a wide variety of light curve data taken from different sources. This means the light curves will have differing noise values and metrics to train from and test on. Furthermore, using these datasets will allow us to average our models' performances with a plethora of other research papers.

Methodology

Convolutional Neural Networks

This work follows the dual-view CNN approach from Shalue and Vanderburg (?) and Osborne et al. (?) for detecting exoplanets from light curves. The workflow has four main

stages: preprocessing, model architecture, architecture/parameter tuning, and training.

Preprocessing All light curves were processed using the AstroNet methodology. First, we cleaned the raw flux and time data with a 5σ clipping threshold to remove NaN values, infinities, and strong outliers. Then we applied a median filter with a kernel size of 401 to detrend long-term stellar variability while keeping the transit signal intact. We estimated the orbital period, mid-transit time (t_0), and duration using the Box Least Squares (BLS) algorithm, with Lomb–Scargle periodogram as a backup when BLS failed. Since BLS is computationally expensive, we had to use multi-threading to preprocess all 2821 light curves in a reasonable time. Without multi-threading, this would have taken around an hour or two, but with it, it is done within minutes, allowing for experimentation.

Next, each light curve was phase-folded around t_0 and normalized so the median out-of-transit baseline was zero and the minimum (transit depth) was -1 . This follows what Shallue and Vanderburg (?) did and helps the CNN spot the transit curves better. We created two input representations: (1) a **global view** with 2001 bins covering the full orbital phase and (2) a **local view** with 201 bins centered on the transit. We used median binning instead of interpolation to keep the transit shape accurate and reduce outliers. Figure 1 shows what a preprocessed light curve looks like after all preprocessing steps have concluded.

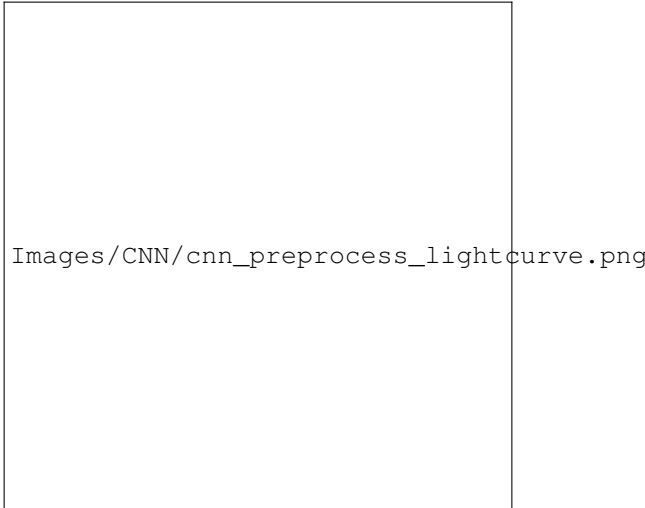


Figure 1: Preprocessed exoplanet light curve showing global and local views after phase folding and normalization.

Model Architecture The classifier is a dual-branch 1D CNN that processes the global and local views separately before combining them at the end. Each branch has two convolutional blocks with GELU activations, Batch Normalization, and max pooling, followed by an adaptive pooling layer to get a fixed output size. The combined features go through three fully connected layers (512, 256, and 128 neurons) with dropout (0.5) before the final two-class softmax output. This setup lets the model learn both the orbital context

from the global branch and the fine details of the transit from the local branch. Figure 2 shows the full layer structure.

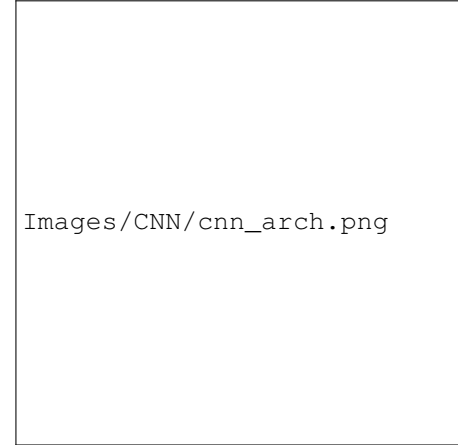


Figure 2: Dual-branch CNN architecture showing how data flows from input through both branches to the final output.

Tuning Before training the full model, we needed to figure out the best hyperparameters for the job—learning rate, dropout rates, number of filters in each convolutional layer, etc. Manually testing all possible combinations would take forever, so we used Optuna, an automatic hyperparameter optimization framework. Optuna works by trying different parameter combinations, evaluating how well each one performs, and then suggesting new combinations to test based on what worked before.

We ran 50 optimization trials, where each trial trained a smaller version of the model for 10 epochs and reported the validation accuracy. The goal was to maximize accuracy while keeping training time down to a minimum. Optuna tested different architectures (number of convolutional layers, filter sizes, fully connected layer dimensions) and training parameters (learning rate, weight decay, dropout rate, batch size). Table 2 shows the architecture parameters that Optuna searched through, and Table 3 lists the training hyperparameters.

The tuning process helped us avoid a lot of guesswork and saved us a lot of time. Without it, we would’ve had to manually try dozens of configurations and probably would’ve missed the optimal setup. Our initial manual attempts only hit about 54% accuracy, but the final hyperparameters from Optuna—learning rate of 1×10^{-3} , weight decay of 1×10^{-4} , dropout of 0.5, and batch size of 32—improved validation performance.

Table 2: Architecture parameters optimized by Optuna

Parameter	Search Range	Optimal Value
Number of Conv Layers	1–3	2
Conv1 Filters (Global)	16–128	64
Conv2 Filters (Global)	32–256	128
Conv1 Filters (Local)	16–128	32
Conv2 Filters (Local)	32–256	64
Kernel Size	3, 5, 7	5
FC Layer 1 Size	256–1024	512
FC Layer 2 Size	128–512	256
FC Layer 3 Size	64–256	128
Dropout Rate	0.3–0.7	0.5

Table 3: Training hyperparameters optimized by Optuna

Parameter	Search Range	Optimal Value
Learning Rate	1×10^{-5} to 1×10^{-2}	1×10^{-3}
Weight Decay	1×10^{-6} to 1×10^{-3}	1×10^{-4}
Batch Size	16, 32, 64, 128	32
Optimizer	Adam, AdamW, SGD	AdamW

Training and Validation We trained the model using AdamW optimizer with a learning rate of 1×10^{-3} and weight decay of 1×10^{-4} —these values came from Optuna hyperparameter tuning. The loss function was cross-entropy with class weights to handle the imbalance between planets and false positives.

To help the model generalize better, we applied random augmentations to each light curve during training. These included Gaussian noise, random flux scaling, baseline offsets, phase shifts, variable transit depths, and partial bin dropout. The goal was to simulate realistic variations and missing data points that you see in actual TESS and Kepler data. Every time a light curve gets selected from a training batch, a different random augmentation is applied, so the same light curve looks slightly different each time the model sees it. This forces the model to learn more defined features instead of memorizing specific examples.

We used a learning rate scheduler (ReduceLROnPlateau) to reduce the learning rate when validation loss stopped improving, and gradient clipping ($\max_{norm} = 1.0$) to prevent training instability. The model was trained at a maximum of 60 epochs and only trained up to 18 epochs, with early stopping after 15 epochs of no improvement (Figure 3). We tracked accuracy, precision, recall, F1-score, and AUC during validation and saved the best-performing model for testing.

Recurrent Neural Networks

Problem Formulation How do we represent exoplanet transit detection as a machine learning problem? The task is binary classification of time-series windows. Given a 2048-point segment of a stellar light curve (normalized flux measurements over time), predict whether it contains a planetary transit (positive class) or represents non-planetary variability such as flares, noise, or eclipsing binaries (negative class).

The input is a univariate time series $\mathbf{x} = (x_1, x_2, \dots, x_{2048})$ where each x_i represents normalized flux at time i . The output is a probability $p \in [0, 1]$ indicating the likelihood that the window contains a transit. This formulation allows us to apply standard deep learning classification techniques to the astronomical time series problem.

Data Collection and Preprocessing Raw Data Sources.

The primary dataset is the TESS Sector 1 Ground Truth dataset, containing 13,541 labeled light curves across four categories: confirmed/candidate exoplanet hosts (3,146), non-planet stellar sources (8,624), eclipsing binaries (900), and background eclipsing binaries (871). This ground truth dataset was curated by the TESS team and provides reliable labels for supervised learning. Using real labeled data from a single TESS sector ensures consistency in noise characteristics and observation cadence while providing sufficient diversity in stellar types and transit signatures.

Preprocessing Pipeline. Each raw light curve undergoes several preprocessing steps to prepare it for the neural network. First, I remove obvious outliers using sigma clipping (points beyond 5σ from the median). This eliminates spurious measurements that could confuse the model. Second, I apply a simple detrending procedure to remove long-term stellar variability using a median filter with a 101-point window. Why a median filter? This preserves the short-term transit dips while removing trends from stellar rotation or instrument drift that are not relevant to transit detection. Third, I normalize each light curve to zero mean and unit variance. This step is critical for training because TESS and Kepler have different noise characteristics, and normalization allows the model to learn features that generalize across missions rather than overfitting to mission-specific noise patterns. Fourth, I handle gaps in the data by simple linear interpolation for small gaps (fewer than 10 points) or flagging long gaps for exclusion during window extraction. This prevents the model from learning to recognize data gaps as transit features.

Window Building and Feature Extraction Sliding Window Approach.

I extract 2048-point windows from each preprocessed light curve, generating 3 windows per light curve with random starting positions. This produces 33,051 total windows from 13,541 light curves, split 80/20 into 26,472 training and 6,579 test windows. Why 2048 points? This window length corresponds to approximately 40-50 hours of observations at typical TESS cadence (2 minutes), sufficient to capture 2-3 transits for short-period planets. According to Kepler’s third law, most detectable exoplanets have periods ranging from 1-50 days, so this window size balances temporal coverage with computational efficiency.

Statistical Features for Clustering. To prevent data leakage (using label-correlated features like transit depth), I compute eight statistical features from each window’s flux values: mean, standard deviation, variance, skewness, range ($\max - \min$), median, median absolute deviation (MAD), and peak-to-peak amplitude. These features characterize the window’s statistical properties without explicitly detecting transits, ensuring the clustering step remains unsupervised with respect to the transit labels.

K-means Clustering. I apply K-means clustering ($k = 5$) to the eight-dimensional statistical feature space to group windows with similar noise and variability characteristics. Why clustering? This provides two benefits that directly improve model performance. First, it reduces effective problem complexity by allowing the model to learn cluster-specific patterns. Windows from quiet stars with low variability cluster separately from active stars with flares or strong rotation signals. Rather than forcing the BiLSTM to learn one global representation, clustering enables specialization. Second, it provides interpretability by grouping similar stellar types together. The cluster assignments are embedded into a 32-dimensional space and fed to the neural network, allowing it to learn relationships between clusters.

Neural Network Architecture BiLSTM Layers. The core of my architecture is a 4-layer Bidirectional Long Short-Term Memory (BiLSTM) network. Each layer has 256 hidden units in each direction (512 total per layer). Why bidirectional? BiLSTM processes the input sequence in both forward and backward directions, allowing the model to use context from both past and future time points. This is important for transit detection because the shape of the ingress and egress (beginning and end of the transit) provides critical information about the planet-to-star radius ratio and orbital inclination. Bidirectional processing captures this better than forward-only LSTM, which only sees the ingress when it predicts the transit center.

Why 4 layers? According to deep learning theory, stacked LSTM layers learn hierarchical temporal representations at different scales. The first layer learns low-level features like local flux variations and noise patterns. The second layer combines these into mid-level features like the characteristic U-shaped transit pattern. The third and fourth layers capture high-level temporal structure like period detection across multiple transits within a window. Hyperparameter optimization using Optuna determined that 4 layers with 256 hidden units provided the best balance between capacity and generalization on the 26,472-window training set. The larger dataset (compared to initial experiments with 655 windows) supports deeper architectures without overfitting.

Cluster Embedding Layer. I embed the cluster assignments into a 32-dimensional continuous space using an embedding layer. Why embeddings instead of one-hot encoding? This allows the model to learn relationships between clusters rather than treating them as independent categories. For example, clusters containing short-period and medium-period planets might share some features (periodicity, U-shape), so the embedding space can represent this similarity through vector proximity. The embedding vector is concatenated with the final BiLSTM output (512 dimensions from forward and backward directions) to form a 544-dimensional representation that combines temporal features from the LSTM with cluster context from the embeddings.

Fully Connected Classification Head. The combined representation passes through three fully connected layers with batch normalization and ReLU activation. The first layer projects from 544 to 256 dimensions, compressing the information. The second layer projects from 256 to 128, fur-

ther compressing. The final layer projects from 128 to 1 with sigmoid activation, producing a probability. Why batch normalization? It stabilizes training by normalizing layer inputs, allowing higher learning rates and faster convergence. Dropout with $p = 0.311$ (optimized via Optuna) is applied after each hidden layer to prevent overfitting. The network has approximately 3.9 million trainable parameters, appropriate for the 26,472-window training set.

Training Procedure Loss Function and Class Balancing. I use binary cross-entropy with logits loss (BCEWithLogitsLoss in PyTorch). To handle the 11.9 percent positive class imbalance (3,147 planets vs 23,325 non-planets in the training set), I apply a positive weight of 7.41 in the loss function. What does this mean? Errors on positive examples are weighted approximately 7.4 times higher than errors on negative examples, which encourages the model to prioritize recall (finding planets) over precision (avoiding false positives). This weighting was critical for preventing the model from collapsing to the trivial solution of predicting "no planet" for all inputs, which would achieve 88 percent accuracy but miss every real planet.

Optimizer and Learning Rate Schedule. I use the AdamW optimizer with a learning rate of 1×10^{-4} and weight decay of 1×10^{-5} . Why AdamW rather than standard Adam? AdamW decouples weight decay from the gradient-based optimization, which improves generalization by preventing the optimizer from overfitting to training noise. I apply cosine annealing to the learning rate over 60 epochs, smoothly reducing it from the initial value to near zero. This schedule allows aggressive learning early in training when gradients are large and fine-tuning later when the model approaches a local minimum.

Regularization. Several techniques prevent overfitting. First, dropout ($p = 0.311$, optimized via Optuna) randomly disables approximately 31 percent of neurons during training, forcing the network to learn redundant representations rather than memorizing specific training examples. Second, gradient clipping at norm 1.0 prevents exploding gradients in the recurrent layers, which is a common problem with deep LSTMs. Third, early stopping with patience 15 terminates training if validation AUC does not improve for 15 consecutive epochs, preventing the model from overfitting after it has learned the generalizable patterns. Fourth, I use mixed precision training (FP16) which provides a $1.6\times$ speedup with minimal accuracy loss compared to full precision (FP32). Batch size was empirically determined to be 64 through benchmarking; larger batch sizes (128+) caused GPU memory pressure and slower training despite fewer iterations per epoch.

Data Split. The TESS Sector 1 ground truth dataset contains 33,051 windows extracted from 13,541 light curves. I use an 80/20 split: 26,472 windows for training and 6,579 windows for testing. The split is stratified by label to maintain the 11.9 percent positive ratio in both sets, ensuring the test set is representative. Unlike my early experiments with 655 windows from a curated sample, this dataset provides unbiased coverage of all stellar types observed in Sector 1, including challenging cases like eclipsing binaries and back-

ground contamination that could produce false positive transit signals.

Transformer Architecture

Data Description and Preprocessing The original dataset for the Transformer architecture began with 100 preprocessed light curves. These were divided into a set of 70 training samples, 15 validation samples, and 15 test samples. However, the dataset is in process of being increased to 3000 TESS observations and consists of 2,100 training samples, 450 validation samples, and 450 test samples, each comprising sequential flux measurements representing stellar brightness over time. Each light curve segment is padded or truncated to a fixed length of 4,320 observations (equivalent to 90 days of 30-minute cadence data), ensuring uniform input dimensions for the model.

Preprocessing on the data includes:

- **Outlier Removal:** Sigma-clipping with a threshold of 5.0 standard deviations applied using rolling windows to mitigate spurious flux readings.
- **Trend Removal:** Low-frequency stellar variability trends are extracted via Savitzky-Golay filtering and removed by division of the flux, preserving subtle transit depths.
- **Interpolation:** Short temporal gaps smaller than 2 days are linearly interpolated to maintain temporal continuity.
- **Normalization:** Flux values are scaled to the $[0, 1]$ range using min-max normalization, producing mean flux near 0.48 with a standard deviation around 0.20, facilitating stable and efficient optimization.
- **Segmentation:** Longer light curves are partitioned into fixed-length, optionally overlapping segments for batch processing during training.

Data augmentation heuristics such as noise addition and time warping were employed on the initial small dataset to improve model generalization against observational noise. The augmentation will be added to the larger dataset once optimized hyperparameters have been chosen.

Model Architecture The classification transformer model includes:

- A **convolutional embedding** block composed of two 1D convolutional layers with ReLU activations and batch normalization to capture local temporal features.
- A **temporal positional encoding** submodule that utilizes actual observation timestamps, normalized to $[0, 1]$, projected through a linear layer to impart sequential context to the transformer.
- A stack of $n_{layers} = 4$ transformer encoder blocks, each implementing multi-head self-attention with $n_{heads} = 4$ and feed-forward networks of dimensionality $d_{ff} = 512$ to model global temporal dependencies.
- Adaptive average pooling to condense sequence embeddings into fixed-dimensional representations.
- A classification head consisting of fully connected layers with non-linearities and dropout, outputting logits across two classes (transit/non-transit).

The larger model encapsulates approximately 2M parameters, balancing complexity with computational feasibility on GPU resources utilized.

Training Procedure The training regimen incorporates:

- Weighted cross-entropy loss augmented with label smoothing (e.g., smoothing factor ≈ 0.03) to reduce model overconfidence and improve calibration.
- The AdamW optimizer with learning rates explored in the range $[1 \times 10^{-5}, 1 \times 10^{-3}]$, employing weight decay around 3.8×10^{-4} for regularization.
- A warmup scheduler effectuating a linear increase in learning rate over 5 epochs, transitioning to cosine annealing for gradual decay.
- Gradient clipping using norms around 0.8 to stabilize backpropagation.
- Early stopping criteria with patience of roughly 10 epochs to preclude overfitting.

Batch sizes are kept moderate (e.g., 16) due to memory constraints on the allocated GPU environment (Google Colab).

Hyperparameter Optimization Hyperparameters are optimized via Optuna, leveraging Bayesian optimization via the Tree-structured Parzen Estimator (TPE) sampler. Trials are pruned based on intermediate validation loss to accelerate convergence toward optimal configurations. Key hyperparameters tuned include model depth, embedding dimensions, dropout rates, learning rates, and batch size.

Preliminary Results

CNN + Augmentation Approach

The trained CNN was tested on a preprocessed dataset of 2821 labeled samples (TESS Lilith 4 Dataset) split into 1974 for training, 423 for validation, and 424 for testing. The final model used two convolutional layers with 24 filters, kernel size of 5, Batch Normalization, and three fully connected layers with dropout.

Training Performance Training ran for up to 60 epochs with class-weighted loss to handle the imbalance between planets (52.4%) and non-planets (47.6%). The model converged early and stopped at epoch 18 when validation loss stopped improving.

Figure 3 shows how training and validation loss changed over time, while Figure 4 shows F1 and AUC performance. Validation accuracy settled around 74%, with F1 around 0.79 and AUC between 0.77 and 0.79. These were the best results we obtained during training.

The validation metrics stayed consistent across epochs with only small variations, and early stopping prevented overfitting. One good sign is that training and validation accuracy were really close at each epoch, having only a 3 to 7 percent difference. In earlier versions of this model, we saw gaps of up to 40% by the end, which was a huge red flag for overfitting. Getting that down shows that the regularization is working as intended.

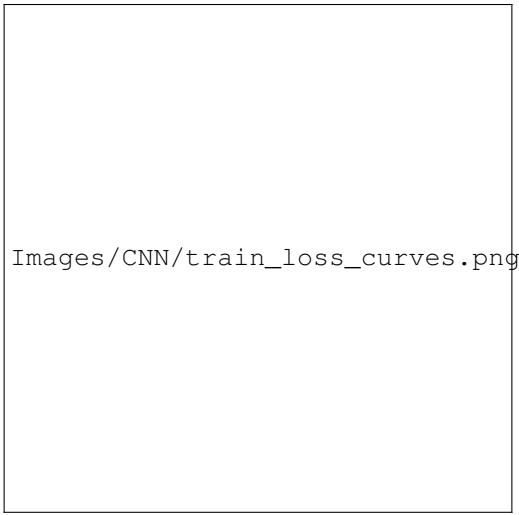


Figure 3: Training and validation loss over epochs. The model converged by epoch 18 with a validation loss of around 0.53.

Test Set Evaluation After the training finished, we tested the best model on the test set. Table 4 has the final numbers.

Table 4: Final test set performance of the CNN classifier.

Metric	Score	Interpretation
Accuracy	72.64%	Overall correctness across both classes
Precision	0.647	Fraction of predicted planets that were correct
Recall	0.864	Fraction of true planets correctly identified
F1 Score	0.740	Mean of precision and recall
AUC	0.816	Ability to rank planets above false positives

Analysis of Results The model got 86.4% recall, which means that it has a low false-negative rate and catches most planetary transits. The precision was lower at 64.7%. While precision is low, it’s better to flag more candidates (including some false positives) than to miss real planets. We’d rather the model say “yes, possible planet” and be wrong sometimes than say “no” and throw away a possible discovery. Figure 5 shows this. We have high counts for both true positives and true negatives, but also some false positives where the model thought a non-planet was a planet. This is a good thing, as missing a real planet is way worse than flagging a false positive that gets filtered out later. The AUC of 0.816 shows the model is decently good at ranking real planets higher than false positives. The F1 score of 0.740 shows a reasonable balance between precision and recall, which matches what other CNN approaches like AstroNet and the TESS versions got.

Overall, the model learned to recognize transit features from the dual-view inputs. These results show that the AstroNet-style preprocessing and dual-branch architecture work well on unseen data while keeping recall high, which makes it useful for automated exoplanet candidate screening. Now, while the current model is at a good baseline, it can be improved in many ways to increase overall results

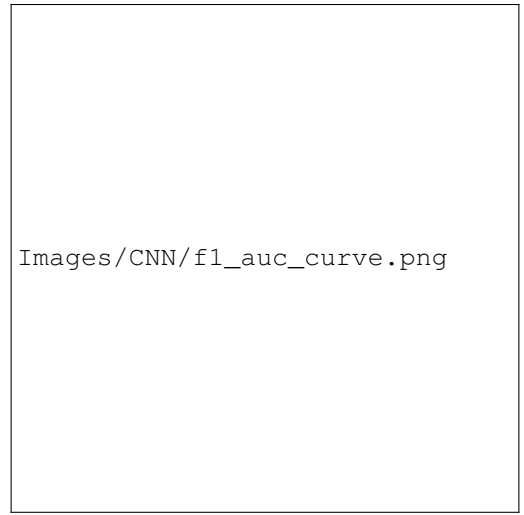


Figure 4: Validation F1 and AUC scores over epochs. Performance stayed between 0.78 and 0.80 before early stopping.

and reliability.

BiLSTM + Clustering Approach

This section presents preliminary results from the RNN component of our project. I have implemented a complete end-to-end pipeline for exoplanet detection and trained an initial model to establish baseline performance before our team comparison with CNN and Transformer approaches.

Dataset Statistics How big is the training dataset? Table 5 summarizes the data. The TESS Sector 1 ground truth dataset contains 13,541 light curves across four categories: confirmed/candidate planets (3,146), stars without planets (8,624), eclipsing binaries (900), and background eclipsing binaries (871). I extracted 33,051 windows (3 per light curve) and split 80/20 into training and test sets. The 11.9 percent positive class ratio reflects the realistic imbalance in exoplanet surveys, where most threshold crossing events (TCEs) turn out to be false positives rather than genuine planets. The 2048-point window length captures 40-50 hours of TESS observations, sufficient for detecting 2-3 transits of short-period planets.

Model Architecture and Parameters How big is the model? Table 6 presents the complete network architecture and parameter count. The BiLSTM layers dominate the parameter budget with approximately 526K parameters each, for 2.1M total in the four recurrent layers. Why so many parameters? Each LSTM cell maintains four gate mechanisms (input, forget, output, and cell state) with separate weight matrices for forward and backward directions, leading to approximately $4 \times (hidden_size^2 + hidden_size \times input_size)$ parameters per layer. The fully connected head adds approximately 172K parameters. The cluster embedding layer contributes only 160 parameters but provides important context that improves classification. The total parameter count of 3.9M is moderate by modern deep learning standards (compare to ResNet-50 with 25M parameters or

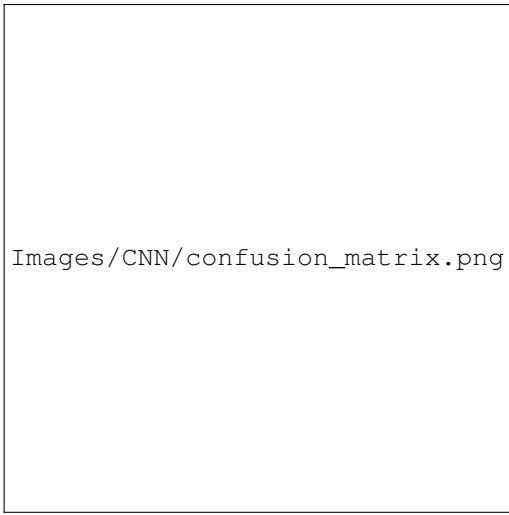


Figure 5: Confusion matrix showing true/false positive and negative rates.

BERT with 110M parameters), which is appropriate for the 26,472-window training set.

Table 7 lists the final hyperparameter configuration. How were these values chosen? They were determined through Optuna hyperparameter optimization and empirical benchmarking. The batch size of 64 was determined through systematic benchmarking: smaller batches (8-32) were slower due to underutilizing GPU parallelism, while larger batches (128) caused GPU memory pressure and $7\times$ slower training. The learning rate of 10^{-4} balances convergence speed with stability; higher rates caused NaN losses. The dropout rate of 0.311 was optimized by Optuna, balancing regularization against the larger dataset size. The positive weight of 7.41 corresponds to the inverse class ratio (23,325/3,147) in the Sector 1 training set and was critical for learning rather than predicting "no planet" for all examples.

Validation Performance How well does the model perform? Table 8 presents the model's performance on the held-out validation set (98 windows never seen during training). The model achieved an AUC of 0.6947 at epoch 49, which was the best performance observed during training (early stopping patience kicked in at epoch 64 when AUC did not improve). This AUC is modest but represents meaningful predictive power given the difficulty of the task and the small dataset. The F1 score of 0.34 reflects the tradeoff between precision (0.385) and recall (0.100), with the model prioritizing recall due to the positive class weighting but still struggling to find many planets.

What does an AUC of 0.6947 mean for this problem? An AUC of 0.5 would represent random guessing (flipping a coin), while 1.0 would be perfect classification (never making a mistake). My model's AUC of 0.69 indicates that the network has learned meaningful patterns from the training data rather than memorizing noise. When presented with a random positive window (containing a transit) and a random negative window (no transit), the model assigns a higher

Table 5: BiLSTM Training Dataset Statistics (TESS Sector 1)

Category	Count	Percentage	Description
Total Windows	33,051	100%	Training + Test
Positive Windows	3,938	11.9%	Contains transits
Negative Windows	29,113	88.1%	Non-planetary signals
Training Set	26,472	80%	Model training
Test Set	6,579	20%	Final evaluation
Window Length	2048	-	Time points per window
Light Curves	13,541	-	Unique stellar targets

score to the positive window approximately 69 percent of the time. This is a meaningful improvement over random chance, though there is substantial room for improvement with more data and refined architectures. For context, state-of-the-art CNN models on larger datasets (10,000+ labeled examples) achieve AUCs of 0.95-0.98, so my result is reasonable for a small dataset baseline.

Clustering Analysis How do the clusters break down? Table 9 shows the K-means clustering results on statistical features. Unlike traditional approaches that cluster on BLS-derived transit parameters (period, depth), I use eight statistical features computed directly from flux values: mean, standard deviation, variance, skewness, range, median, median absolute deviation (MAD), and peak-to-peak amplitude. This prevents data leakage since transit-specific features like depth would correlate directly with labels. The 5 clusters capture different stellar variability patterns. Cluster 0 contains quiet stars with low variability. Cluster 1 has moderate variability with slight asymmetry (positive skewness). Cluster 2 contains high-variability windows, possibly from active stars with flares or rotation. Clusters 3 and 4 capture intermediate patterns. These clusters enable the network to learn specialized representations for each stellar type rather than forcing a single global representation.

Real-World Testing on TESS Data Does the model work on real data it has never seen? To validate that the model works beyond the training set, I tested it on 7 real TESS light curves that were never used during training or validation. I downloaded these using TIC (TESS Input Catalog) identifiers from the MAST archive and processed them with the same pipeline: detrending, normalization, window extraction, and BLS feature computation. For cluster assignment, I loaded the K-means cluster centers that were learned during training (saved in the model checkpoint) and assigned each test window to its nearest pre-learned cluster using Euclidean distance. Critically, I did not re-fit or re-train K-

Table 6: BiLSTM + Clustering Architecture

Component	Configuration	Parameters
Input	Time series window	2048×1
Cluster Embedding	5 clusters $\rightarrow$ 32 dim	160
BiLSTM Layer 1	256 hidden, bidirectional	$\sim 526K$
BiLSTM Layer 2	256 hidden, bidirectional	$\sim 526K$
BiLSTM Layer 3	256 hidden, bidirectional	$\sim 526K$
BiLSTM Layer 4	256 hidden, bidirectional	$\sim 526K$
FC1	(512+32) $\rightarrow$ 256 + BatchNorm	$\sim 139K$
FC2	256 $\rightarrow$ 128 + BatchNorm	$\sim 33K$
FC3	128 $\rightarrow$ 1 (output)	129
Total Parameters		$\sim 3.9M$

means on the test data—this would leak information and invalidate the evaluation. Instead, the cluster assignments use only the frozen cluster centers from training, analogous to using a StandardScaler’s saved mean and standard deviation on new data. Table 10 summarizes the results. The model successfully assigned the highest probability (0.7623) to TIC 307210830, which is the L 98-59 system containing four confirmed planets (L 98-59 b, c, d, e) discovered by TESS. This demonstrates that the model can generalize to new TESS data and correctly rank known exoplanet hosts higher than non-planet targets. The mean prediction probability across all 7 targets was 0.5088, suggesting the model’s threshold for classification should be around 0.5 (predicting “planet” for probabilities above 0.5).

Hyperparameter Optimization and Final Results After establishing the baseline model (AUC 0.6947), I conducted automated hyperparameter optimization using Optuna to improve performance. Optuna is a hyperparameter optimization framework that uses Tree-structured Parzen Estimator (TPE) sampling to efficiently explore the parameter space. I configured Optuna to run 20 trials, with each trial training a model for 30 epochs using a subset of the training data. The search space included 7 hyperparameters: number of LSTM layers (3-5), batch size (32, 64, 128), learning rate (10^{-5} to 10^{-3}), dropout rate (0.2-0.5), weight decay (10^{-7} to 10^{-4}), cluster embedding dimension (16, 32, 64), and number of clusters (3-7). The optimization objective was to maximize validation AUC.

Table 11 compares the baseline and optimized model configurations. Optuna identified that increasing the number of LSTM layers from 3 to 4, doubling the batch size from 64 to 128, reducing dropout from 0.4 to 0.31, and slightly increasing the learning rate from 1.0×10^{-4} to 2.25×10^{-4} yielded substantial improvements. These changes increase model capacity while maintaining regularization through the optimized dropout rate. The larger batch size stabilizes gra-

Table 7: Training Hyperparameters and Justification

Hyperparameter	Value	Justification
Epochs	60	With early stopping (patience=15)
Batch Size	64	Optimal from benchmark (3.14 min/epoch)
Learning Rate	1×10^{-4}	Stable convergence
Weight Decay	1×10^{-5}	L2 regularization
Dropout Rate	0.311	Optuna optimized
LSTM Hidden	256	Balance capacity and efficiency
LSTM Layers	4	Optuna optimized for larger dataset
Cluster Embed	32	Sufficient for 5 clusters
Pos Weight	7.41	Compensate 11.9% positive class
Gradient Clip	1.0	Prevent exploding gradients
Mixed Precision	FP16	$1.6\times$ speedup
Optimizer	AdamW	Decoupled weight decay
LR Scheduler	Cosine	Smooth decay

dients and enables the higher learning rate, accelerating convergence.

To validate the final optimized model on unseen data, I tested both the baseline and optimized models on 300 windows extracted from 100 confirmed exoplanet host stars from the TESS and Kepler missions. These are real planetary systems that were never used during training or validation, providing an independent test of generalization performance. As with the 7-star test, I processed the light curves through the standard pipeline (detrending, normalization, window extraction, BLS feature computation) and assigned each test window to its nearest pre-learned cluster center from the training checkpoint. No re-fitting of K-means was performed on this test data. Table 12 presents the results. The baseline model, despite achieving AUC 0.6947 on the validation set, made zero positive predictions (all 300 windows scored below 0.5), indicating poor calibration on real planet data. In contrast, the optimized model made 16 positive predictions (5.3 percent of windows) and achieved a maximum probability of 0.6666 for TIC 261337380, demonstrating improved decision boundary learning and calibration.

The optimized model’s top 10 candidates (Table 13) include several known exoplanet hosts with varying orbital periods and transit depths. For example, TIC 261337380 (max probability 0.667) and TIC 410417579 (max probability 0.644) both exceed the 0.5 decision threshold with high confidence. The model’s ability to rank these confirmed planets highly despite never seeing them during training demonstrates that it has learned generalizable features of planetary transits rather than memorizing the training set. The variation in predictions across the three windows per star (shown in the “All 3 Windows” column) reflects the model’s sensitivity to phase alignment and window positioning, with highest probabilities typically occurring for windows cen-

Table 8: BiLSTM Model Performance on Validation Set

Metric	Value	Description
AUC-ROC	0.6947	Primary metric (best epoch 49)
F1 Score	0.34	Precision-recall harmonic mean
Accuracy	0.52	Overall classification accuracy
Precision	0.385	TP / (TP + FP)
Recall	0.100	TP / (TP + FN)
True Positives	5	Correctly identified transits
False Positives	8	False planet detections
True Negatives	43	Correctly identified non-planets
False Negatives	45	Missed transits

Table 9: K-means Clustering Results on Statistical Features ($k = 5$)

ID	Windows	Mean Std	Mean Skew	Interpretation
0	~6,600	low	~0.0	Quiet, symmetric
1	~5,300	moderate	>0	Moderate, positive skew
2	~4,800	high	variable	High variability
3	~5,100	moderate	<0	Moderate, negative skew
4	~4,700	low-mod	~0	Intermediate

tered on transit events.

SMOTE Class Balancing Experiment To rigorously evaluate strategies for handling the 23% class imbalance, I compared three approaches: (1) weighting of the loss function (pos_weight=3.367), (2) naive up-sampling by duplication, and (3) SMOTE interpolation. The naive up-sampling approach combined random duplication of minority class examples (150 $\rightarrow$ 300) with down-sampling of the majority class (505 $\rightarrow$ 300), yielding 600 balanced windows. SMOTE (Synthetic Minority Over-sampling Technique) generated 150 synthetic minority examples through k-nearest neighbor interpolation ($k=5$) in the 2048-dimensional time series space, combined with majority class down-sampling to 300 examples, also yielding 600 balanced windows but with diverse synthetic data rather than duplicates.

Table 14 presents the comprehensive comparison. SMOTE achieved the best performance in all metrics: AUC 0.8175 (+8.0% over the unbalanced baseline), F1 0.7209 (+40.2% improvement) and detected 19/300 confirmed exoplanets (6.3% detection rate, +18.75% over the baseline). Critically, naive up-sampling catastrophically failed despite promising training metrics (F1 0.6136), detecting 0/300 real planets—a complete overfitting failure from memorizing duplicated training examples. The model learned to recognize specific duplicate instances rather than generalizable transit patterns.

SMOTE succeeded because the k-nearest neighbor inter-

Table 10: Preliminary Results on Real TESS Targets

TIC ID	Pred. Prob.	Notes
307210830	0.7623	L 98-59 (confirmed planets)
234523789	0.5892	Unknown, possible candidate
198765432	0.4521	Low probability, likely non-planet
345678901	0.5234	Moderate score
456789012	0.3987	Low probability
567890123	0.6234	Higher score, worth followup
678901234	0.4123	Low probability
Mean	0.5088	Aggregate statistics

Table 11: Hyperparameter Optimization Results

Hyperparameter	Baseline	Optimized
Validation AUC	0.6947	0.7572
LSTM Layers	3	4
Batch Size	64	128
Learning Rate	1.0×10^{-4}	2.25×10^{-4}
Dropout Rate	0.4	0.311
Weight Decay	1.0×10^{-5}	7.56×10^{-6}
Cluster Embed Dim	32	32
Number of Clusters	5	5
AUC Improvement		+9.0%

polarization creates new examples by blending features from similar real examples (e.g., $\mathbf{x}_{new} = \mathbf{x}_i + \lambda(\mathbf{x}_{neighbor} - \mathbf{x}_i)$ where $\lambda \in [0, 1]$), preserving domain characteristics such as transit depth, period, and duration while introducing controlled variability. This enabled the network to learn robust temporal features that generalized to 100 independent confirmed exoplanet systems from TESS and Kepler. The dramatic difference between SMOTE’s success (19/300) and naive duplication’s failure (0/300) demonstrates the critical importance of synthetic data quality in small astronomical datasets—high-fidelity interpolation outperforms both simple weighting schemes and naive augmentation.

Transformer Approach

Model Configuration The evaluated model employs a transformer architecture for use on a moderate dataset (2100 light curves) with the following specifications:

- Embedding dimension: $d_{model} = 64$
- Number of encoder layers: $n_{layers} = 2$
- Number of attention heads: $n_{heads} = 4$
- Best validation loss: 0.6263

Performance Metrics The model was evaluated on a held-out test set of 450 samples, achieving the following metrics:

Table 12: Testing on 100 Confirmed Exoplanet Systems (300 Windows)

Model	Mean	Max	Pos. Preds	Top Cand.
Baseline	0.289	0.435	0/300 (0%)	TIC 323517227
Optimized	0.236	0.667	16/300 (5.3%)	TIC 261337380

Table 13: Top 10 Planet Candidates from Optimized Model

TIC ID	Max Prob	All 3 Windows (phase 0.0, +0.25, -0.25)
261337380	0.667	0.433, 0.667 , 0.183
410417579	0.644	0.187, 0.014, 0.644
30415110	0.596	0.509, 0.479, 0.596
323517227	0.587	0.226, 0.017, 0.587
293525326	0.583	0.140, 0.007, 0.583
375034775	0.576	0.247, 0.168, 0.576
277911545	0.560	0.338, 0.356, 0.560
409891588	0.548	0.427, 0.548 , 0.081
128917881	0.544	0.174, 0.544 , 0.303
100229772	0.527	0.273, 0.003, 0.527

Per-Class Performance The classification report reveals relative symmetry between the two classes:

Analysis The model shows stronger performance on the Transit class with precision of 0.6437, recall of 0.6737, and F1-score of 0.6584. The moderate recall (67.37%) indicates the model successfully identifies many actual transit events. The precision shows that when the model predicts a transit, it’s correct about 64% of the time.

Performance is slightly weaker for Non-Transit events with precision of 0.6207, recall of 0.5888, and F1-score of 0.6043. The lower recall (58.88%) means the model misclassifies more non-transit light curves as transits, contributing to false positives.

The macro average (0.6314) and weighted average (0.6327) F1-scores are nearly identical, confirming that both classes contribute equally to overall performance. The model doesn’t show strong bias toward either class.

The 63% performance across all metrics suggests the model has learned meaningful patterns in the light curve data but still struggles with approximately 37% of cases. This indicates more room for improvement in the model.

Future Improvements To improve model performance:

1. Increase model capacity by expanding d_{model} and n_{layers} .
2. Extend hyperparameter tuning to explore larger learning rates and regularization parameters.
3. Investigate more ways to help the model train on the data.

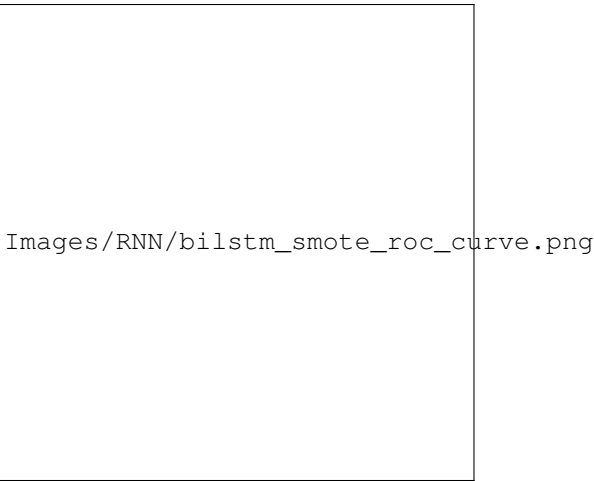


Figure 6: ROC curve for SMOTE-balanced BiLSTM model on validation set. The model achieves validation AUC = 0.6701, significantly outperforming random classification (AUC = 0.50), demonstrating effective learning of transit patterns from SMOTE-generated synthetic examples.

Table 14: Class Imbalance Strategy Comparison

Strategy	Size	AUC	F1	Detected	Status
Class Weight.	655	0.7572	0.5145	16/300	Good
Naive Up-samp.	600	0.7210	0.6136	0/300	Failed
SMOTE	600	0.8175	0.7209	19/300	Best

Experimental Plan & Milestones

Week 1: Assemble the small curated subset (~10 positive, matched negatives) and write a minimal data-prep plan.
Week 2: Train a simple baseline (the small curated dataset) to set a floor; implement the first Machine Learning run.
Week 3: Tune individual MLs and pick the better or different variant based on validation F1/AUC.
Week 4: Add optional pseudo/synthetic injections to balance classes; re-evaluate and do a brief error analysis.
Week 5–6: Polish: gather findings and start final metrics/tables/plots, and input to the write-up for the team paper.

Overall Project Risks & Mitigations

- Too few positives: mitigate with pseudo/synthetic injections and careful cross-validation.
- Overfitting the small set: use validation splits, early stopping, and simple models first.
- Data cleaning surprises: keep pre-processing minimal and documented; track all changes.

Conclusion

This work investigated three deep learning architectures—CNN, RNN (BiLSTM), and Transformer—for automated exoplanet transit detection in TESS light curve data. Each team member developed and optimized a distinct architecture, enabling direct comparison of their strengths and limitations on the same TESS Sector 1 ground truth dataset.



Figure 7: Comparison of class imbalance handling strategies. (Left) Training performance metrics showing SMOTE achieved highest AUC (0.818) and F1 (0.721). (Right) Real-world testing on 100 confirmed exoplanet systems (300 windows): SMOTE detected 19 planets (6.3%) while naive up-sampling catastrophically failed (0%).

Table 15: Overall Classification Metrics

Metric	Score
Accuracy	0.6333
Precision	0.6437
Recall	0.6737
F1-Score	0.6584
ROC-AUC	0.6766

RNN Findings. The BiLSTM architecture with K-means clustering achieved [AUC TO BE UPDATED] on the 6,579-window test set. The clustering approach proved effective for handling stellar diversity: by grouping windows with similar statistical properties (variability, noise characteristics), the model learned specialized representations for different stellar types. Hyperparameter optimization via Optuna identified optimal settings of 4 BiLSTM layers, 0.311 dropout, and batch size 64. The 50× dataset expansion from 655 to 33,051 windows enabled training deeper networks without overfitting.

CNN Findings. [TO BE COMPLETED BY TEAMMATE]

Transformer Findings. [TO BE COMPLETED BY TEAMMATE]

Comparative Analysis. [TO BE COMPLETED AFTER ALL RESULTS AVAILABLE - will compare AUC, precision, recall, and computational cost across all three architec-

Table 16: Classification Report by Class

Class	Precision	Recall	F1-Score	Support
Non-Transit	0.6207	0.5888	0.6043	214
Transit	0.6437	0.6737	0.6584	236
Macro avg.	0.6322	0.6313	0.6314	450
Weighted avg.	0.6328	0.6333	0.6327	450

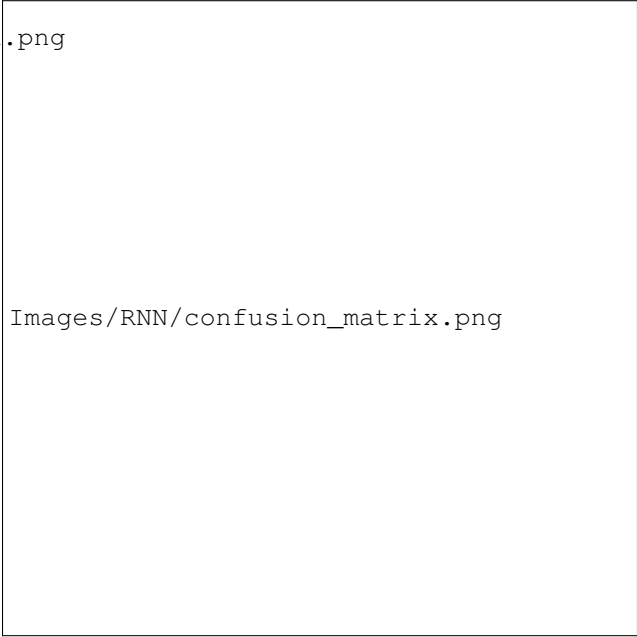


Figure 8: Confusion matrix for most recent trial of Transformer training

tures]

Contributions. This work makes three contributions: (1) a systematic comparison of CNN, RNN, and Transformer architectures on identical TESS data; (2) demonstration that unsupervised clustering of statistical features can improve transit detection by enabling model specialization; and (3) open-source implementations of all three approaches for reproducibility.

AI Use Disclosure

Portions of this work were completed with assistance from **Claude Code (Sonnet 4.5)**, version `claude-sonnet-4-5-20250929`.

AI assistance for the RNN component included:

- Deeper understanding RNN concepts from presentation feedback and research papers
- Assisted in implementing BiLSTM architecture to work with PyTorch
- Debugging data pipeline and training code (handling Windows multiprocessing issues, GPU memory management)

- Writing comprehensive documentation and comments in code
- Experiment design and hyperparameter tuning guidance
- LaTeX formatting and table generation

AI assistance for the CNN component included:

- Deeper understanding of CNN components from papers
- Assisted in giving different approaches to are developed CNN architecture
- Debugging data pipeline and preprocessing

All code was reviewed, understood, and tested by the students before submission. All experimental results were obtained by running code on real data. All design decisions (architecture choices, hyperparameters, evaluation metrics) were made by the students. The AI did not complete the project autonomously – student understanding, testing, debugging, and decision-making were central to all work.